

Deep Learning Practical 2: Model answers

Foundational mathematics

Recap of foundational calculus

Derivatives

Plot $f(x) = \tanh x$, its first derivative and its second derivative.

Differentiating gives

$$f'(x) = 1 - \tanh^2 x, \quad f''(x) = -2 \tanh x (1 - \tanh^2 x).$$

As f rises smoothly from -1 to 1 , its steepest slope occurs at the origin, where $f'(0) = 1$. Towards either tail the curve flattens and f' approaches zero. The second derivative is positive to the left of zero and negative to the right, which describes the change from bending upwards to bending downwards.

Rules of differentiation

What is the derivative of $h(x) = \sin(x^2)$?

Applying the chain rule to the outer sine and the inner square gives

$$h'(x) = \cos(x^2) \frac{d}{dx} x^2 = 2x \cos(x^2).$$

Partial derivatives

Calculate the gradient of $f(x, y, z) = \cos x + z^3 - x^2 y$.

When differentiating with respect to one coordinate, hold the other two fixed. This gives $\partial f / \partial x = -\sin x - 2xy$, $\partial f / \partial y = -x^2$ and $\partial f / \partial z = 3z^2$, which form the column gradient

$$\nabla f = \begin{bmatrix} -\sin x - 2xy \\ -x^2 \\ 3z^2 \end{bmatrix}.$$

Rules of partial differentiation

Differentiate $z^4 - \sin(y^2 + x)$ with respect to y , then x and z .

The inner function $y^2 + x$ contributes a factor of $2y$ when differentiating with respect to y , and a factor of 1 when differentiating with respect to x . The term z^4 is constant in both calculations, so

$$\frac{\partial}{\partial y} (z^4 - \sin(y^2 + x)) = -2y \cos(y^2 + x), \quad \frac{\partial}{\partial x} (z^4 - \sin(y^2 + x)) = -\cos(y^2 + x).$$

With respect to z , the sine term is constant and the derivative is $4z^3$.

Jacobian matrix

Differentiate the helix map with respect to its two inputs.

For $f(t, s) = (\sin t + s, \cos t, 6t/\pi)^\top$, the columns correspond to changes in t and s respectively:

$$J_f(t, s) = \begin{pmatrix} \cos t & 1 \\ -\sin t & 0 \\ 6/\pi & 0 \end{pmatrix}.$$

Changing s moves the output only along its first coordinate, whereas changing t moves it around the helix and along its vertical axis.

Computational graphs and backpropagation

Differentiate the affine layer and the squared-error loss.

An affine layer $f(x) = Wx + b$ has Jacobian W . Writing its prediction error as $e = Wx + b - t$, the loss is $L = e^\top e$, so the chain rule gives

$$\nabla_W L = 2ex^\top, \quad \nabla_b L = 2e, \quad \nabla_x L = 2W^\top e.$$

These expressions use column vectors for x , b and e . They follow from $\nabla_e(e^\top e) = 2e$, with the outer product $ex^\top$ providing one weight derivative for each output–input pair.

Programming exercise: learning two spirals

Data generation

Sketch the shapes of `arm`, `X` and `y`. Which axis does `stack` create, and which axis does `cat` extend? What shape would `model(X) - y` have if `y` were `(256,)`?

`arm` has 128 rows and two coordinate columns. Stacking the two coordinate vectors creates that column axis, while concatenating the `arm` with its reflection extends the row axis to give `X` shape `(256, 2)`. There is one label per row, so `y` has shape `(256, 1)`.

If `y` instead had shape `(256,)`, broadcasting against the `(256, 1)` predictions would produce a `(256, 256)` array. This would compare each prediction with every label, rather than pairing each point with its own label.

Building the network

Calculate the number of parameters in each Linear layer. Which operations have no learned parameters?

Layer	Calculation	Parameters
$2 \rightarrow 32$	$2 \times 32 + 32$	96
$32 \rightarrow 32$	$32 \times 32 + 32$	1,056
$32 \rightarrow 2$	$32 \times 2 + 2$	66
$2 \rightarrow 1$	$2 \times 1 + 1$	3
Total		1,221

Each `tanh` applies a fixed function without introducing any learned parameters. For hidden width d , the same calculation gives $d^2 + 6d + 5$ parameters.

Calculating gradients

Use `model[:1](X)` to check the printed derivatives. Has `backward()` changed the weights? Explain the need to clear gradients.

Since $s_i = \sum_j w_j h_{ij} + b$, differentiating the mean squared error gives

$$\frac{\partial L}{\partial w_j} = \frac{2}{N} \sum_i (s_i - y_i) h_{ij}.$$

The matrix product below evaluates both output-weight derivatives together.

```
model.zero_grad()
score = model(X)
loss = ((score-y)**2).mean()
loss.backward()
H = model[:1](X).detach()
manual = 2*(score.detach()-y).T @ H / len(X)
torch.testing.assert_close(model[-1].weight.grad, manual)
```

Calling `backward()` stores derivatives in `.grad` while leaving the weights unchanged. The subsequent subtraction updates the weights. Because PyTorch adds each new derivative calculation to any existing gradients, we clear them before calculating the next update.

Training and inspecting the predictions

Where do the colours still disagree, and when does the fit improve most quickly?

Early in training, the broad colour regions cut across the spiral arms, so many dots sit on a background of the wrong colour. As the boundary bends around the arms, these disagreements retreat to the regions where the spirals are close together. The fastest improvement occurs during the steep falls of the loss curve as the network learns this separation. Later updates make smaller adjustments to scores that already have the correct sign.

Checking new observations: experiments

1. Remove all three Tanh operations. Predict the separation and compare the predictions and loss with the original network.

Composing affine maps produces another affine map, $s(x) = Ax + c$. Its zero boundary is therefore a straight line, which cannot wind between these spirals. In the supplied comparison the resulting MSE is 0.93179 and held-out accuracy is 60.06%, compared with MSE 0.003436 and 100% accuracy for the nonlinear width-32 network. More linear layers cannot supply the missing curvature.

2. Try widths 8 and 16. Compare learning speed, final loss, held-out accuracy and parameter count. Can equal accuracies accompany different losses? How small can the network be while preserving a good fit?

Using the sheet's data seed 42, weight seed 7 and 6,000 full-batch updates at learning rate 0.1 gives the following CPU results. Learning speed is measured by the first update at which training MSE falls below 0.01.

Width	Parameters	Final MSE	Held-out accuracy	Updates to MSE < 0.01
2	21	0.734250	71.00%	—
4	45	0.352875	84.47%	—
6	77	0.006886	100%	5,298
8	117	0.004403	100%	4,627
16	357	0.005273	100%	4,623
32	1,221	0.003436	100%	3,918

Widths 8 and 16 reach the threshold at almost the same time, while width 32 reaches it earlier. Width 6 is the smallest successful model among these trials, achieving a good fit with only 77 parameters.

Accuracy depends only on whether the score has the correct sign. Squared error also penalises its distance from the target $+1$ or -1 , which explains how networks with identical classifications can have different losses. A successful small model demonstrates that its width suffices for this run, but does not establish the smallest possible width across initialisations and training times.

Visualising gradient descent

1. Predict the output when both new weights are zero. How does the contour plot explain the direction of the first update?

With $w_1 = w_2 = 0$, every point receives the fixed output bias, approximately 0.030 in the supplied run. The first update follows the negative gradient, which is perpendicular to the local contour and points towards smaller loss.

2. Change the learning rate from 0.2 to 0.02 and then 0.8, restarting from zero. Compare the paths and final losses after 20 updates.

After 20 updates, the supplied runs give MSEs of 0.38568, 0.003473 and 0.003436 for learning rates 0.02, 0.2 and 0.8 respectively. At 0.02 the weights travel only part of the way towards the minimum, whereas 0.8 reaches approximately its final loss within seven steps. With 0.2, the final weights are about (1.278, 0.225). Increasing the step size further can overshoot the minimum repeatedly and cause divergence, as occurs at 1.2 in this example.

3. Why can two adjustable weights now separate the spirals, while removing every Tanh prevented this?

These two weights combine features produced by the trained nonlinear hidden layers. A straight boundary in that learned feature space can correspond to a curved boundary in the original coordinate plane. Removing tanh instead makes the whole mapping affine in the original coordinates.

Discussion points and open research questions

Why do we often need thousands of optimisation steps? Is spending millions on training mathematically necessary?

A gradient describes local behaviour, so one large step cannot generally follow a curved route to a useful solution. Minibatches also reveal only part of the data at each update. Repeated updates let the network adjust its features as it encounters more examples. Neither a particular step count nor a monetary cost follows as a mathematical necessity: both depend on the task, available data, model, training method and desired quality.

How would batch size influence the number of gradient-update steps required?

At a fixed number of epochs, a batch of size B gives about N/B updates per epoch. At a fixed update count, increasing B instead processes more examples. Larger batches reduce the noise from sampling examples, but this does not imply a proportional reduction in updates to reach a chosen accuracy, because the learning rate and optimisation behaviour also matter.

Could we learn to store datasets in 1–10 optimisation steps? How do compression and generalisation differ?

For a suitable dataset and model, rapid memorisation is possible, especially with a useful pretrained representation or a direct fitting method. It is not a general guarantee for arbitrary datasets under gradient descent. Compression concerns how many bits are stored, including weights and any per-example codes, at a given reconstruction error. Generalisation concerns performance on new observations from the intended distribution, so storing the training examples accurately does not by itself demonstrate generalisation.